```
import numpy as np
import cv2
import matplotlib.pyplot as plt
import random
import math

np.random.seed(42)
random.seed(42)

size = 256
y, x = np.ogrid[:size, :size]

brain = (
    (x - 128)**2 / 90**2 +
    (y - 128)**2 / 110**2
) <= 1

image = np.zeros((size, size), dtype=np.uint8)

noise = np.random.normal(0, 8, (size, size))

image[brain] = np.clip(
    85 + noise[brain], 0, 255
)

roi = (
    (x - 145)**2 / 25**2 +
    (y - 125)**2 / 20**2
) <= 1

image[roi] = np.clip(
    190 + noise[roi], 0, 255
)

reference = roi.astype(np.uint8)

num_nests = 10
iterations = 20
pa = 0.25
alpha = 0.5
beta = 1.5

def dice_score(predicted, reference):

    intersection = np.sum(predicted * reference)

    total = np.sum(predicted) + np.sum(reference)

    if total == 0:
        return 1.0

    return (2 * intersection) / total


def fitness(threshold):

    predicted = (
        image >= threshold
    ).astype(np.uint8)

    return 1 - dice_score(predicted, reference)


def levy_flight():

    sigma = (
        math.gamma(1 + beta) *
        math.sin(math.pi * beta / 2) /
        (
            math.gamma((1 + beta) / 2) *
            beta *
            2 ** ((beta - 1) / 2)
        )
```

```
    ) ** (1 / beta)

    u = np.random.normal(0, sigma)
    v = np.random.normal(0, 1)

    return u / (abs(v) ** (1 / beta) + 1e-10)


nests = np.random.uniform(0, 255, num_nests)

scores = np.array([
    fitness(nest) for nest in nests
])

best_index = np.argmin(scores)

best_nest = nests[best_index]
best_score = scores[best_index]

for iteration in range(iterations):

    for i in range(num_nests):

        new_nest = (
            nests[i] +
            alpha * levy_flight() *
            (nests[i] - best_nest)
        )

        new_nest = np.clip(new_nest, 0, 255)

        new_score = fitness(new_nest)

        j = random.randint(0, num_nests - 1)

        if new_score < scores[j]:

            nests[j] = new_nest
            scores[j] = new_score

    abandon_count = int(pa * num_nests)

    worst_indices = np.argsort(scores)[-abandon_count:]

    for i in worst_indices:

        nests[i] = np.random.uniform(0, 255)
        scores[i] = fitness(nests[i])

    best_index = np.argmin(scores)

    if scores[best_index] < best_score:

        best_score = scores[best_index]
        best_nest = nests[best_index]

    print(
        "Iteration:", iteration + 1,
        "Best Dice Score:", round(1 - best_score, 4)
    )


optimal_threshold = int(round(best_nest))

segmented_image = (
    image >= optimal_threshold
).astype(np.uint8)

final_dice = dice_score(
    segmented_image,
    reference
)

print("\nOPTIMIZED SEGMENTATION RESULTS")
print("Optimal Threshold:", optimal_threshold)
```

```
print("Dice Similarity Coefficient:", round(final_dice, 4))
print("Segmentation Accuracy:", round(final_dice * 100, 2), "%")


plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title("Original MRI Image")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(reference, cmap='gray')
plt.title("Reference Segmentation")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(segmented_image, cmap='gray')
plt.title("CS Optimized Segmentation")
plt.axis('off')

plt.tight_layout()
plt.show()
```

```
Iteration: 1 Best Dice Score: 1.0
Iteration: 2 Best Dice Score: 1.0
Iteration: 3 Best Dice Score: 1.0
Iteration: 4 Best Dice Score: 1.0
Iteration: 5 Best Dice Score: 1.0
Iteration: 6 Best Dice Score: 1.0
Iteration: 7 Best Dice Score: 1.0
Iteration: 8 Best Dice Score: 1.0
Iteration: 9 Best Dice Score: 1.0
Iteration: 10 Best Dice Score: 1.0
Iteration: 11 Best Dice Score: 1.0
Iteration: 12 Best Dice Score: 1.0
Iteration: 13 Best Dice Score: 1.0
Iteration: 14 Best Dice Score: 1.0
Iteration: 15 Best Dice Score: 1.0
Iteration: 16 Best Dice Score: 1.0
Iteration: 17 Best Dice Score: 1.0
Iteration: 18 Best Dice Score: 1.0
Iteration: 19 Best Dice Score: 1.0
Iteration: 20 Best Dice Score: 1.0

OPTIMIZED SEGMENTATION RESULTS
Optimal Threshold: 137
Dice Similarity Coefficient: 1.0
Segmentation Accuracy: 100.0 %
```

